Grid-LLM: A Distributed SDN and Large-Language-Model Framework for Contextual DNP3 Defense

Mentor: Harshavardhan Chintapatla
Project members: Adelaide Aboagye, Amrutha Varshini Bolli, Wen-Hsin Chen, Krishna Rithwik Karra, Sai Srikar Reddy Kolli, Varsha Kuchana, Sai Divya Reddy Malyala, Varshith Kumar Pulluri

Abstract — Smart Grids are connected power grids that enable grid manageability. However, the increased connectivity of power grids introduces new vulnerabilities. The objective of this research was to engineer an architecture to detect and mitigate a Master Impersonation attack targeting Distributed Energy Resources (DERs) via Distributed Network Protocol (DNP3) based on the groundwork laid out by previous research[1]. The proposed architecture consists of a distributed Software-Defined Networking (SDN) testbed, an attack detection component on the testbed, and a Large Language Model Analysis within the attack mitigation component.

I. PROBLEM DEFINITION

The need to power electronic technologies has grown dramatically over the last century as these utilities have become ubiquitous in households, workplaces, and infrastructure. To meet the rising demand for electricity, modern power grids require systems that can learn and respond to fluctuating electricity demand in real time [2]. Smart grids provide this capability by using digital technologies to remotely monitor and control grid operations.

Historically, security was never a prioritized design consideration for Supervisory Control and Data Acquisition (SCADA) systems because they operated in isolation with dedicated, separate communication links [3, 4]. Today, with the rise of increasingly interconnected smart grids, the Distributed Network Protocol (DNP3) has become the primary protocol used in the US energy sector to facilitate reliable data communication across these systems [5]. Operators or automated processes use DNP3 to send control commands, known as function codes, from a utility master in the control center to outstations in the field. Although efficient, a publication found that DNP3 is susceptible to 28 generic attack vectors [6] which could allow attackers to gather reconnaissance, disturb, or even remotely take over grid management.

A real-world example of such a vulnerability occurred on December 23rd, 2015, when three Ukrainian power distribution companies sustained a highly-coordinated cyberattack. Initially the attackers infiltrated the IT network through a spear-phishing attack deploying a malicious malware suite known as BlackEnergy. Once inside, the malware spread to the Operational Technology (OT) system including the SCADA platforms that manage distribution networks. The attackers used this access to remotely disable electrical substation by switching off breakers [7], causing power outages for approximately 230,000 customers in Western Ukraine for one to six hours. This attack demonstrated how SCADA systems and DNP3 protocol

vulnerabilities can be exploited and the serious threat a compromised smart grid poses to public safety and critical infrastructure.

Literature reviews reveal that legacy DNP3 configurations are vulnerable to spoofing, replay, and control-plane attacks because they lack authentication, encryption, and fine-grained segmentation. Existing Intrusion Detection Systems are either centralized or use switch-based filters which are mitigation strategies that are prone to single-point failure. Thus, building atop of prior research [1], a distributed Software Defined Network (SDN) was implemented into the proposed architecture.

The three research questions that this research aimed to answer were:

- *How to detect DNP3 lateral movement attacks where outstations send unauthorized administrative function codes to another outstation?*
- *How to apply LLM semantic reasoning to understand DNP3 protocol intent and recommend mitigation policies?*
- *How integrate LLM-based threat analysis with SDN controllers to automatically deploy mitigation policies?*

II. CORE LOGIC

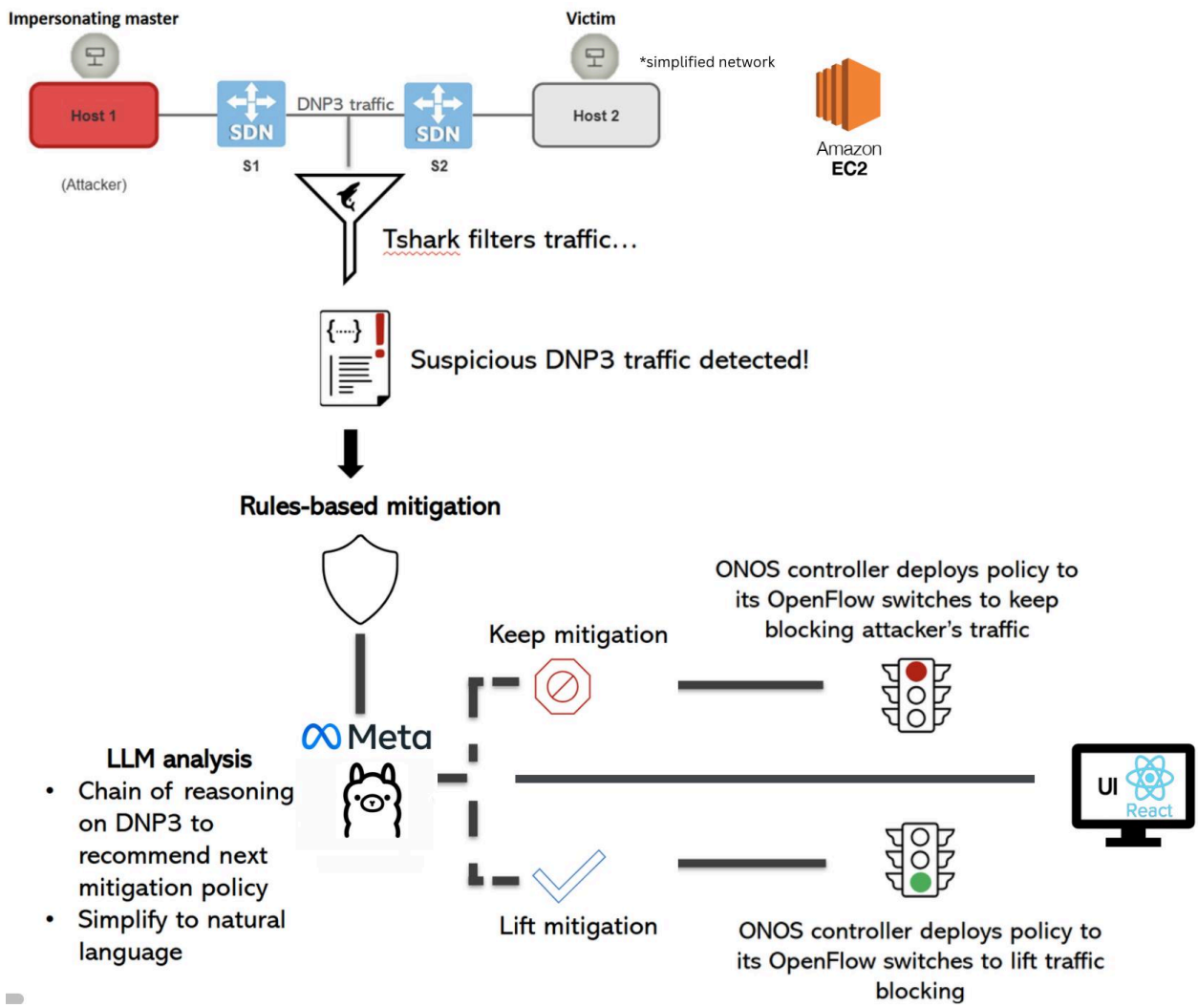


Fig. 1. Architecture

A. *Testbed construction*

A distributed Software-Defined Networking (SDN) architecture was built as a testbed in order to simulate real Smart Grid environments, which must scale across wide-area infrastructures. This design choice also addressed a primary limitation of centralized SDN architectures, where the single root controller can become a bottleneck for geographically distant network segments. In contrast, the distributed SDN architecture allows controllers to enforce policies and direct traffic within their local domains while maintaining a shared global state through state synchronization. Although inter-controller synchronization introduces additional overhead, the overall system achieves a lower response latency due to the reduced distance between controllers and their local switches [8].

Due to the high infrastructural cost of running separate Amazon Elastic Compute Cloud (EC2) instances for the SDN controllers, this research constructed the testbed using one EC2 instance. First, a Docker-based virtual network was instantiated on the EC2 instance. Then, the SDN controllers were containerized and deployed on the network. This approach not only reduced EC2 infrastructure cost but allowed for agile deployment, teardown, and replicability of the experiment while preserving its distributed-control nature.

Atomix was utilized to synchronize the states of all SDN controllers across the distributed system. If a controller was compromised, isolated, or failed, Atomix ensured that the system maintains consistency by preserving the states of the unaffected controllers as the most recent Raft-consensus–agreed version. This provides high availability, avoids total loss of control-plane functionality during cyber incidents, and mitigates the propagation of errors and inconsistencies [9].

Three Open Network Operating System (ONOS) controllers served as the SDNs within the distributed SDN network. After Atomix was set up, the three ONOS controllers were connected to it to establish state consistency among themselves. The ONOS controllers will efficiently deploy traffic blocking policies to the OpenFlow switches during a cyberattack.

Mininet was used to create the distributed SDN network's OpenFlow switch topology. After the SDN setup, the switch topology was constructed on the AWS EC2 instance using a custom topology Python script. The script assigned each ONOS controller with three OpenFlow switches and then assigned each OpenFlow switch to two outstations. The resulting Mininet topology is a hybrid implementation combining mesh and star topologies. The Mininet topology will simulate DNP3 communication between SCADA master and outstation. One host will be selected to represent the Supervisory Control and Data Acquisition (SCADA) master in a smart power grid while the rest will represent the outstations.

The Mininet topology is a hybrid of mesh and star topologies. Each ONOS controller manages its own local star topology of OpenFlow switches. One core OpenFlow switch within the local star topology is connected to an OpenFlow switch from the other two ONOS controllers. These three OpenFlow switches will form a triangular mesh topology. There are many advantages to using the star-mesh hybrid topology. For instance, the mesh topology center ensures failure of one core switch doesn't isolate the whole branch–this removes the single-point failure limitation from pure-star topology [10]. Additionally, the center mesh links can provide alternative routes for traffic between different stars. Thus, the mesh topology center distributes congestion and improves redundancy. The local star topologies make adding and removing devices a simple process that won't require changing the rest of the network. Additionally, faults

are simple to identify within the ONOS controller's local star because the paths have minimal complexity. Thus, the local star topologies improve scalability and simplify management.

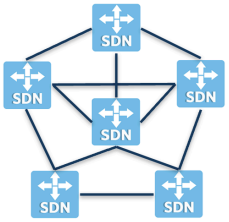


Fig 2. Mesh topology

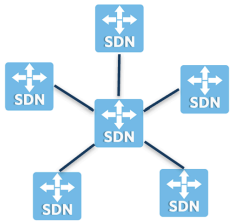


Fig 3. Star topology

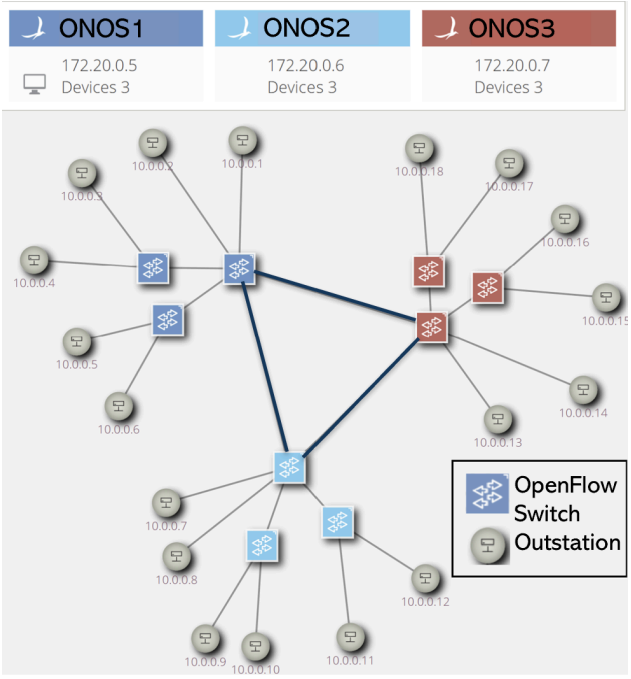


Fig 4. Testbed's star-mesh hybrid topology

B. *Master Impersonation Attack Detection*

Master impersonation is a critical lateral-movement attack in DNP3-based SCADA networks where a compromised outstation illegitimately assumes the role of a master and issues unauthorized administrative commands to peer outstations. Because legacy DNP3 was designed without authentication or role verification, outstations inherently trust any device capable of sending valid DNP3 packets over the control network. This makes master impersonation attacks particularly dangerous and difficult to detect using traditional signature-based intrusion detection mechanisms.

The detection mechanism operates within a distributed Software-Defined Networking (SDN) testbed that emulates real-world smart grid communication patterns. Live DNP3 traffic is continuously monitored at the network edge using packet inspection, focusing on application-layer fields critical for attack identification. The extracted parameters include source and destination IP addresses, TCP port numbers, DNP3 application-layer function codes, sequence numbers, internal indication flags, and unsolicited response indicators.

Detection logic specifically targets administrative DNP3 function codes—such as cold restart, warm restart, stop application, delete file, and unsolicited responses—that are expected to originate exclusively from the legitimate master node. When such privileged commands are observed originating from an outstation, the behavior is flagged as suspicious.

To improve accuracy and reduce false positives, captured packets are temporarily buffered within a short correlation window and grouped into command–response sequences. This grouping provides contextual awareness of lateral movement behavior, ensuring that isolated packets or incomplete sessions are not incorrectly classified as attacks. Each grouped event is then evaluated by a machine-learning classifier, which confirms whether the behavior corresponds to a master impersonation attack and identifies the specific type of administrative command being abused.

Once an attack is confirmed and verified as non-duplicate, a structured alert is generated containing the attacker identity, victim identity, command type, timestamps, and packet-level evidence. This alert is forwarded to the mitigation framework for immediate containment and further analysis.

C. *Attack mitigation*

How LLM and rules-based mitigation works?

To achieve both rapid response and high-confidence decision-making, the proposed system employs a hybrid two-layer mitigation strategy that combines rule-based SDN enforcement with Large Language Model (LLM)–driven semantic analysis.

Phase 1: Rule-Based Mitigation

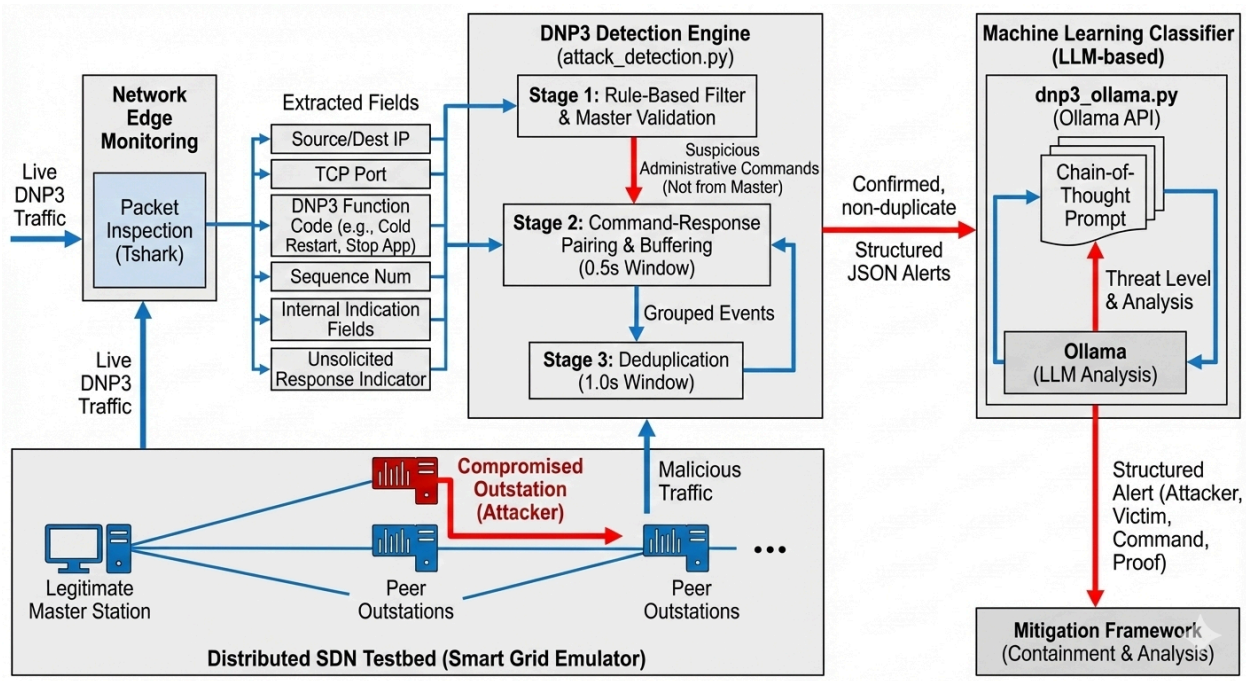
Upon detection of a confirmed master impersonation attack, the SDN controller immediately enforces a mitigation policy at the data plane. This policy installs flow rules on OpenFlow switches to block all traffic originating from the attacking outstation toward other devices in the network. The rule-based response provides near-instant containment, preventing further propagation of malicious commands and minimizing the risk of cascading failures in the smart grid.

This phase operates independently of higher-level analysis to ensure low latency and deterministic behavior, which is essential for protecting time-critical industrial control systems.

Phase 2: LLM-Based Verification and Explainability

In parallel with the immediate containment, the attack alert is submitted to a Large Language Model for semantic reasoning. The LLM analyzes the grouped DNP3 events, command semantics, attacker–victim relationships, and operational context to determine whether the detected activity aligns with legitimate maintenance operations or represents a true malicious intrusion.

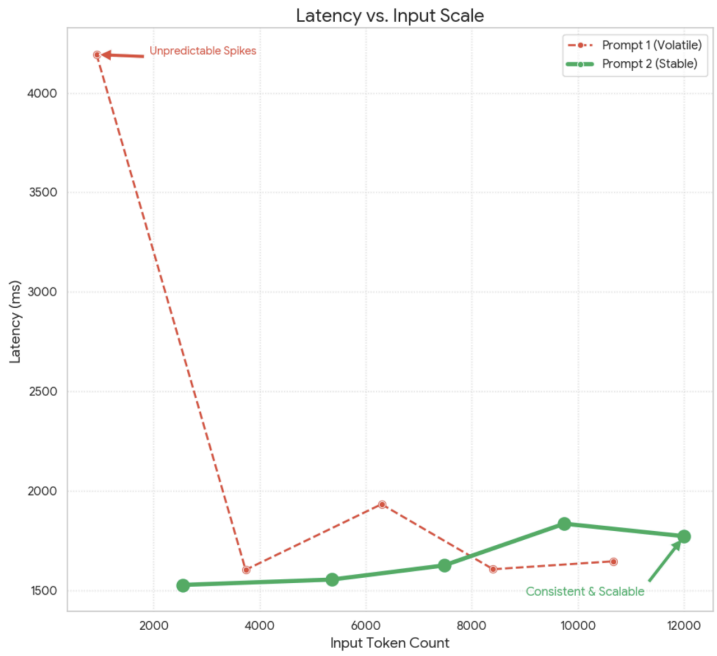
Using structured reasoning steps, the LLM evaluates the severity and potential impact of the attack and produces a human-readable explanation of its assessment. Based on this analysis, the LLM recommends whether the mitigation policy should be maintained or lifted. If the activity is confirmed as malicious, the blocking rules remain enforced; if the activity is deemed legitimate, the SDN controller removes the temporary restrictions.



Analysis and Latency results:

The collected data consists of structured attack logs and system-generated outputs obtained during detection and mitigation experiments. Analyzing this data helps identify attack patterns, frequency of events, and system behavior under different conditions. This analysis is essential for understanding whether the detection pipeline captures meaningful security signals and whether the reasoning and mitigation components operate as intended.

Prompt 1 in the graph is zero-shot, and prompt 2 is a chain of thought strategy. Both prompts were evaluated using structured attack JSONs generated by the attack_detection.py module to ensure realistic and consistent experimental conditions. Latency measurements indicate that Prompt 1 experiences unpredictable response-time spikes as input size increases. Prompt 2 maintains stable and predictable latency, scaling smoothly with larger inputs. The structured reasoning in Prompt 2 limits unnecessary output generation and reduces computational overhead, resulting in more consistent performance.



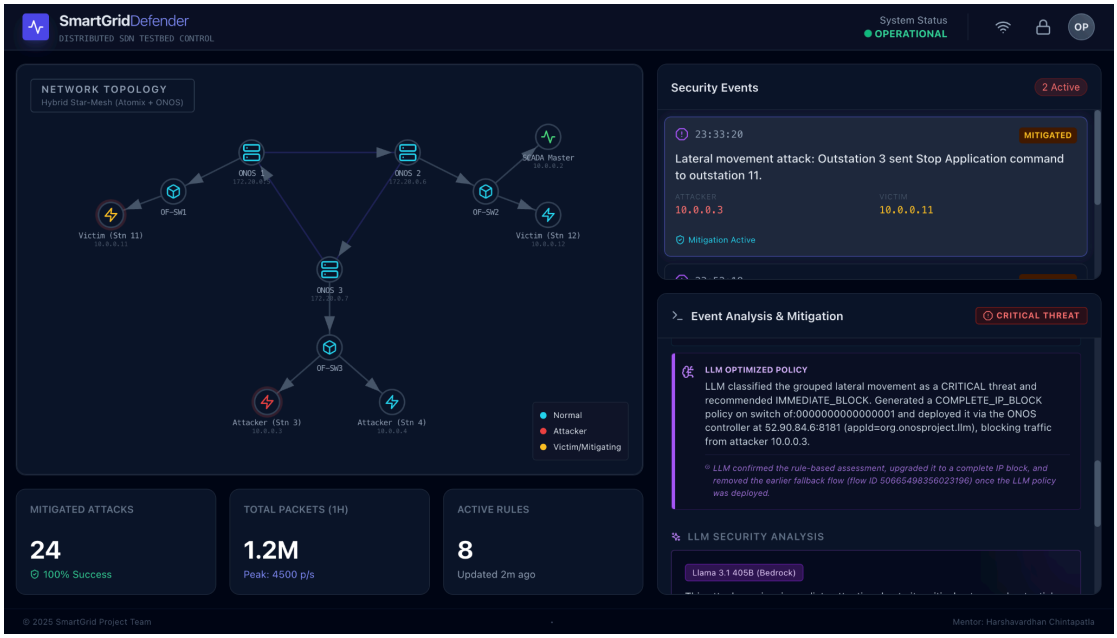
D. *User Interface*

Rubric requirements:

- simple & intuitive interface snapshots
- demo of implemented capabilities
- story of solution validation (how it answers research questions/how were goals completed?)
- Solution advantages/limitations

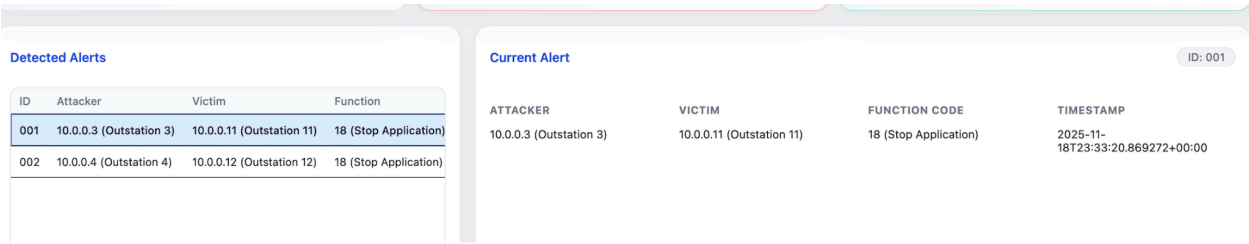
DASHBOARD USER INTERFACE

The Grid-LLM DNP3 Defense Dashboard was developed to provide operators with an intuitive, high-level view of master impersonation attacks and their corresponding SDN and LLM responses. The dashboard is implemented as a web application using a React front-end built with Vite, which communicates with the Node.js/Express backend described in Section II. The backend exposes grouped attack events and associated LLM outputs through REST APIs; the dashboard consumes these endpoints and renders them into an operator-friendly interface rather than raw JSON logs.



The main layout follows a simple overview-and-detail pattern. The top of the interface contains a persistent header with the system title ("Grid-LLM DNP3 Defense Dashboard") and status indicators that show whether the testbed is live, the current deployment mode (e.g., Local Dev), and the active backend API endpoint. Immediately below the header, a set of summary cards provides at-a-glance statistics such as the total number of grouped alerts, the number of critical attacks, and the number of attacks that have been successfully mitigated by the SDN/LLM pipeline. These cards give operators a rapid sense of the system's current security posture without requiring them to inspect individual log entries.

The central portion of the dashboard is split into two primary panels. On the left, a "Detected Alerts" table lists all grouped attack events with columns for alert ID, attacker IP and station, victim IP and station, and the DNP3 function code . Selecting a row in this table drives the rest of the interface, updating the details view for the currently active alert. On the right, the "Current Alert" panel presents the full context for the selected event, including attacker and victim identities, function code, and precise timestamp. This master detail layout allows operators to quickly navigate between alerts while keeping the per-alert context visible.



This user interface directly supports the research goals by making LLM-driven defenses observable and interpretable. For the first research question, grouped lateral-movement attacks detected in the SDN testbed are surfaced as discrete entries in the “Detected Alerts” table, with clear attacker–victim mappings and function codes. For the second question, the “LLM Reasoning” card exposes the semantic understanding of DNP3 intent in plain language, allowing operators to validate whether the model’s interpretation matches operational expectations. For the third question, the “Mitigation Summary” card documents how LLM recommendations translate into SDN flow rules, including which switch received a COMPLETE_IP_BLOCK and how fallback rules were replaced. The dashboard thus provides a simple, intuitive way to validate that the detection logic, LLM analysis, and SDN enforcement layers are working together as intended.

From a usability perspective, the interface adopts a clean, minimal design with consistent typography, spacing, and color cues to reduce cognitive load for operators. Critical alerts and mitigation states are highlighted using accent colors, while the LLM reasoning and packet trace sections use larger fonts and monospace formatting, respectively, to improve readability. However, the current prototype has limitations: it operates on a fixed set of pre-generated alerts rather than a live streaming feed, it does not yet support complex filtering or historical search, and it assumes a single user environment without access control. Future work could extend the dashboard with real-time updates from the detection pipeline, database-backed alert storage, role-based views for operators and analysts, and a network-topology visualization panel that overlays attacks and mitigation rules onto the star-mesh SDN topology.

REFERENCES

[1] V. Pusapati, H. Chintapatla, R. L. Neupane, and P. Calyam, “Distributed Sdn Control for securing DNP3 protocol-based communications in Smart Grids,” *2025 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, pp. 1–7, Sep. 2025. doi:10.1109/smartgridcomm65349.2025.11204599

[2] [1]“Smart Grid,” *Energy.gov*. [Online]. Available: https://www.energy.gov/smart-grid.

[3] C. Feltus, M. Ouedraogo and D. Khadraoui, “Towards cyber-security protection of critical infrastructures by generating security policy for SCADA systems,” Information and Communication Technologies for Disaster Management (ICT-DM), 2014 1st International Conference on, Algiers, 2014, pp. 1–8.

[4] Cyber security risk assessment for SCADA and DCS networks, ISA Trans. 2007 Oct; 46(4): 583–94. pub 2007 Jul 10.

[5] IEEE Standard for Electric Power Systems Communications-Distributed Network Protocol (DNP3) – IEEE Std 1815–2012 (Revision of IEEE Std 1815–2010) -, vol., no., pp. 1,821, Oct. 10 2012.

[6] Z. Drias, A. Serhrouchni and O. Vogel, “Taxonomy of attacks on industrial control protocols,” 2015 International Conference on Protocol Engineering (ICPE) and International Conference on New Technologies of Distributed Systems (NTDS), Paris, 2015, pp. 1–6.

[7] J. Don, “Lessons learned from a forensic analysis of the Ukrainian Power Grid cyberattack,” Lessons Learned From a Forensic Analysis of the Ukrainian Power Grid Cyberattack, 03-Dec-2021. [Online]. Available: https://blog.isa.org/lessons-learned-forensic-analysis-ukrainian-power-grid-cyberattack-malware.

[8] J. Kim *et al.*, “Ambusher: Exploring the security of Distributed Sdn Controllers Through Protocol State Fuzzing,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 6264–6279, 2024. doi:10.1109/tifs.2024.3402967

[9] G. Abdullah Mohammed and O. A. Ibrahim Aldabbagh, “A comparative evaluation of the performance of SDN Controllers (Onos) using Docker Container,” *Technium: Romanian Journal of Applied Sciences and Technology*, vol. 8, pp. 25–31, Mar. 2023. doi:10.47577/technium.v8i.8585

[10] S. Lee, “Smart Grid Network Topology Guide,” *Number Analytics*, 13-Jun-2025. [Online]. Available: https://www.numberanalytics.com/blog/ultimate-guide-network-topology-smart-grids